Kitaptaki problem → Max Sub Sum

Kitaplar → Knuth Algorithm Book serisi (Donald Knuth), Data Structures and Algorithm Analyses in C++ (kitabın kodları kısa kod ile çok iş yapıyor, bizim takip edeceğiniz kitap bu), Robert Sedgewick'in kitabı.

* Machine Learning algoritmalarının (Logistic Regression, Random Forest ...) karmaşıklıkları. ML'de train ve test aşamasındaki karmaşıklıkları ayırıyor.

P → S_1 ⋮ S_n } Time, memory → # command

recursive fibonacci
- → loop → n
- → recursive → 2^n

Time'ı ölçmek için 2 yöntem:
1) number of command
2) pc'ye zamanı yazdırıyoruz

ignoring

Elimizde bir problem var ve bu problem için 1'den fazla çözüm var. Bu çözümlerin time ve memorylerine bakacaz.

Analizde "Time"a bakıyoruz. Şimdilik memory 2. planda.

Zaman Karmaşıklığı Sınıfları ve P/NP Ayrımı

Algoritmaların büyüme hızlarını (growth rate) yani time complexity sınıflarıdır.

- C (Constant)
- $\log n$
- n
- $n \log n$
- n^2
- n^3
- n^4
- ⋮
- 2^n

} P, NP

Polinomsal Zaman (P)
- $O(1)$ → sabit zaman (constant / c)
- $O(\log n)$ → logaritmik
- $O(n)$ → doğrusal
- $O(n \log n)$ → doğrusal - logaritmik
- $O(n^2)$ → kareyel
- $O(n^3)$ → kübik

Genel Form = $O(n^k)$ → polinomsal zaman

↳ Bunlar genellikle deterministik bir algoritma ile makul bir sürede çözülebilir problemler.

← Non-Deterministic Polinomsal Zaman (NP)
- $O(2^n)$

↳ Bu tip karmaşıklıklar üstel, çok hızlı büyür, pratikte büyük n için hesaplanamaz hale gelir.

NP için, çözüm bulmak polinomsal zamanda garanti değildir.

• ML'de karmaşıklık M.N ise M sütun N satır oluyor.

Asimptotik Gösterim

Bunlar büyüme sınırlarını ifade eder.

• O → * if, else, ; (Big-O) • Ω • Θ • o (little-o)

1) Big-O – Üst Sınır (Upper Bound)

• Algoritmanın en kötü durumda en fazla ne kadar büyüdüğünü gösterir.

"Bu algoritma en fazla f(n) kadar büyür." → O(f(n))

Worst-case analiz temsil eder.

* $f(n) \leq c.g(n)$, $n > n_0$ → Bir fonksiyon keyfi bir sabit ile çarpıldığında sürekli büyük çıkıyorsa bu fonksiyona $f(n) = O(g(n))$ diyoruz.

$f(n)$, $g(n)$
n , n^2

$f(n) = O(g(n))$
n , n^2

$n = O(n^2)$
$n = O(n^3)$
$n = O(n^4)$
⋮
$n^5 = O(n^5)$ → Direkt n^5 diyebiliriz. En mantıklısı kendisine en yakın notasyonu almak.
n^6

$n^2 = O(n^2)$ — n^x, $n \log n$ ×
n^3
n^4
⋮
$n^3 = O(n^3)$ → Bunun altındaki ifadeler olmaz.
n^4
⋮
2^n

2) Ω – Alt Sınır (Lower Bound)

• Algoritmanın en az ne kadar süreceğini gösterir.

"Bu algoritma en az f(n) kadar zaman alır" → O(f(n))

Best-case veya teorik alt sınır olarak düşünülür.

* Ω: $f(n) \geq c.g(n)$, $n > n_0$ → Öyle bir fonksiyon var ki onda sürekli aynı seviyede veya küçük oluyor.

$n = \Omega(n)$
$\log n$
c

Direkt eşit olması gerekiyor.

3) Θ - Sıkı Sınır (Tight Bound)

• Hem üst hem alt sınır aynı mertebedeyse kullanılır. Algoritma tam olarak bu büyüme mertebesindedir.

* $f(n) = O(g(n))$
  $g(n) = O(g(n))$
  $\Rightarrow f(n) = \Omega(n)$

$f(n) = \Theta(g(n))$
yani;
$f(n) = O(g(n))$ ve $f(n) = \Omega(g(n))$
(ikisi aynı anda sağlanmalı)

4) little-o - Sıkı Üst Sınır (Strict Upper Bound)

• Daha zayıf bir üst sınırdır.
Algoritma $f(n)$'den daha yavaş büyür.

$x = O(x^2)$

$x = o(x^2)$

$x^2 \neq o(x^2)$
aynı büyüklükte olmuyor.

* $f(n) < c \cdot g(n)$
$f(n) = o(g(n))$

$(\log(k))^n = o(n)$ ... little-o

Örnekler:

• $f(n) = n^2+5$, $g(n) = n+7$ } $f(n) = \Omega(g(n))$ ✓, $f(n) = O(g(n))$ ✗

$n^2+5 \to n^2$ } $n^2 = O(n)$ ✗ , $n^2 = \Omega(n)$ ✓
$n = O(n^2)$ ✓

• $\underbrace{5n^2+6n+3}_{n^2} = O(n^2), n^3, n^4 \ldots$
$\Omega(n^2), \underbrace{n, n\log n, \log n, c \ldots}_{\text{ona kendisine en yakın olanı tercih ediyoruz.}}$

n^2 ymiş gibi düşünüyoruz.

• $5n^2+6n+3$ (vs) $6n^2 \to f(n) = \Theta(g(n))$
$\neq 0$

$5n^2+6n+3 = O(6n^2)$
$5n^2+6n+3 = \Omega(6n^2)$ > $5n^2+6n+3 = \Theta(6n^2)$

$n \log n > n^2$

- $n \log n$, n^2 $\longrightarrow$ $n \log n = \Theta(n^2)$ ✗ → İkisi de aynı seviyede olmalı
  $n \log n = O(n^2)$ ✓
  $n \log n = \Omega(n^2)$ ✗

- $2n^2+5$, $3n^3-20$ → $2n+5 = \Omega(3n^3-20)$ ✗
  $2n+5 = O(3n^3-20)$ ✓
  $2n+5 = \Theta(3n^3-20)$ ✗

* 2 fonksiyon ardışık olarak çağırıldığında toplanır:

$$(\underset{f_1}{n^2} + \underset{f_2}{n^3}) = O(n^3)$$ → 2 algoritmayı birlikte ardışık çalıştırdıktan sonra karmaşıklığı direkt büyük olan alınır.

* 2 fonksiyon iç içe (nested) ise yani bir fonksiyon diğerinin içerisinde çağırılmışsa çarpılır:

$$(\underset{f_1}{n^2} \cdot \underset{f_2}{n^3}) = O(n^5)$$ → Algoritmalar nested (iç içe) ise karmaşıklık üler toplamı şeklinde (çarpım) bulunur.

Hoca 3 dedi bunu

f_1
f_2
toplama

$f_1()$
$f_2()$
çarpma

Her türlü karmaşıklığı büyük olan alınır.

2.4.1. Örnek Algoritma

$1^3 + 2^3 + 3^3 + \ldots + N^3$ toplamını hesaplayan kod:

```
int sum(int n){
    int partialSum; → Değişken tanımlama 0 zaman alır.
    partialSum = 0; → Atama işlemi 1 kez çalışır.
    for(int i=1; i<n; ++i) → 2n+2
        partialSum += i*i*i; → 4n
    return partialSum; → 1 kez çalışır
}
```

* for (int i = 1; i ≤ n ; ++i) → Bu for döngüsünün 3 gizli maliyeti var:

a) Başlatma i=1 → Sadece 1 kez yapılır → 1 işlem

b) Test: i ≤ n kontrolü → Döngü her turda "devam et mi?" diye sorar

- N=3 için: i=1 ✓, i=2 ✓, i=3 ✓, i=4 ✗ (dur)
- Yani N+1 kez test yapılır (son "hayır" testi de sayılır) → N+1 işlem

c) Arttırma: ++i → Her turda i'yi 1 arttırır, N kez yapılır → N işlem

↳ Toplam: $1 + (N+1) + N = 2N+2$

* partialSum += i * i * i

Bu satır döngünün içinde, N kez çalışır. Her çalışmada şu işlemler var:

i * i → 1 çarpma
(i * i) * i → 1 çarpma
partialSum + ... → 1 toplama
partialSum = ... → 1 atama

} Yani her turda 4 işlem, N tur → 4N işlem.

↳ Bütün çalışma toplamları → $6n+4$ → n çok büyük olduğunda +4 ve 6 katsayısı önemsiz olur. Geriye sadece n kalır. Yani karmaşıklık → O(n) olur. (1 tane döngü old. için n oldu.)

↳ Her seferinde bu kadar detaylı saymak gereksiz ve zor. Big-O kullanırken bazı kısayollar var:

- Sabit sayılar (4, 6) yok sayılır.
- Döngü olmayan satırlar önemsizdir.
- Sadece en baskın (en büyük büyüyen) terim önemlidir.

! Yani direkt döngü sayısına bakıp karmaşıklığı belirleyebiliriz.

* N çok büyüdükçe döngü dışındaki her şey küçücük kalır.

Durum	Örnek	Big-O
Döngü yok	sadece atama, return	O(1)
1 döngü	for i=1 to N	O(N)
İç içe 2 döngü	for i, for j içinde	$O(N^2)$
İç içe 3 döngü	for i, for j, for k içinde	$O(N^3)$

Örnek 2.4.1 için:

n defa döndüyor (6 → for döngüsü; for bloğunda çalıştırdığı komut sayısı: 1)

+

$6n+4 \rightarrow O(n)$

} 1 tane döngü old. için O(n) oldu.

Rule 1 - For Döngüleri (For Loops)

Bir for döngüsün çalışma süresi, döngü içindeki işlemlerin süresi çarpı kaç kez tekrarlandığıdır.

```
for (i=0 ; i<n ; ++i)
    ++k ;
```

→ n kez çalışır → $O(n)$

} Döngü içine bak, kaç kez dönüyorsa çarpılır.

↳ for loops → $O(n)$, n

Rule 2 - İç içe Döngüler (Nested Loops)

İçten dışa doğru analiz et. En içteki işlemin süresi, tüm döngülerin boyutlarının çarpımıdır.

```
for (i=0 ; i<n ; ++i)
    for (j=0 ; j<n ; ++j)
        ++k ;
```

→ $n * n = n^2$ kez → $O(n^2)$

} Kaç iç içe döngü varsa, n'leri birbiriyle çarp.

↳ nested loops → $O(m.n)$, m.n

for ... → m → m kez dönüyor
⋮
for ... → n → n kez dönüyor
⋮

} diyelim

Rule 3 - Ardışık (Peş Peşe) Bloklar

Bunlar toplanır, yani en büyük olan kazanır.

```
for (i=0 ; i<n ; ++i)
    a[i] = 0 ;
for (i=0 ; i<n ; ++i)
    for (j=0 ; j<n ; ++j)
        a[i] += a[j] + i + j ;
```

} $O(n)$

} $O(n^2)$

} $\underbrace{O(n) + O(n^2) = O(n^2)}_{\text{Büyük olan değer alınır}}$

↳ consecutive (sequantial)

for ... } n.n
⋮ for
for ... } n

} n^2

Rule 4 – If/Else

```
if(koşul)
    S1
else
    S2
```

Çalışma süresi, S1 ve S2'den hangisi daha uzun sürüyorsa odur. if ve else'ten daha yavaş olan hangisiyse onu al.

↳

```
if (koşul)
    O(n) iş
else
    O(n²) iş
```

En kötü ihtimale göre $O(n^2)$ denir.

↳

```
if
  : → S1
else
  : → S2
```

max(S1, S2)

• Bir if var ya da switch var. Hangisi büyükse onu al. En kötü olana girmek yeterli.

$T(n)$ (n boyutlu problem çözülürken önce n-1 boyutlu problem çözülür sonra sabit maliyeti O(1) bir iş yapılır. $\underbrace{\quad}_{T(n-1)}$

$$T(n) = 1 + T(n-1)$$
$$T(n-1) = T(n-2) + 1$$
$$T(n-2) = T(n-3) + 1$$
$$\vdots$$
$$+\ T(2) = T(1) + 1$$
$$T(n) = n \rightarrow O(n)$$

Telescoping method: Özellikle toplam veya yinelemeli (recurrence) ifadelerini sadeleştirmek için kullanılan matematiksel bir teknikdir. Terimler ardışık olarak birbirini götürür ve toplam büyük ölçüde sadeleşir.

* bir dizideki ardışık elemanlardan oluşan alt dizileri arasında toplamı en büyük olanı bulma problemi için farklı çözümlerin karmaşıklıklarını karşılaştıralım.

↳ Algoritmada deneyeceğimiz dizi "4, -3, 5, -2, -1, 2, 6, -2" olsun. Algoritmaların 11 sonucunu dönmesi gerekir.

↳ Maximum Subsequence Sum Problem

Algoritm 1

* Cubic maximum contiguous subsequence sum algoritm.

```
int maxSubSum1 (const vector<int> & a){
    int maxSum = 0;
    for(int i=0; i<a.size(); ++i)
        for(int j=i; j<a.size(); ++j){
            int thisSum=0;
            for (int k=i; k<=j; ++k)
                thisSum += a[k];
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    return maxSum;
}
```

Bu algoritmada 3 döngü olduğu için karmaşıklık $O(n^3)$'tür.

İç içe 3 döngü

Algoritm 2

* Quadratic maximum contiguous subsequence sum algoritm.

```
int maxSubSum2 (const vector<int> & a){
    int maxSum = 0;
    for (int i=0; i<a.size(); ++i){
        int thisSum=0;
        for(int j=i; j<a.size(); ++j){
            thisSum += a[j];
            if (thisSum > maxSum)
                maxSum = thisSum;
        }
    }
    return maxSum;
}
```

karmaşıklık $O(n^2)$ çünkü 2 tane for döngüsü var.

Algoritm 3

* Rewrsive maximum contiguous subsequence sum algoritm.
* Finds maximum sum in subarray spanning a[left ... right].
* Does not attempt to maintain actual best sequence.

Rewrsive algoritma. Diziyi ikiye bölüp uçlara kadar gidiyor.

karmaşıklığı:

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

$$2 / T(n/2) = 2 \cdot T(n/4) + n/2$$

$$+ \; T(2) = 2 \cdot T(1) + n \rightarrow \text{recursion bitişi}$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = O(n \log n)$$ → Algoritma 3 karmaşıklık olarak daha güzel bir yöntem diğerlerine göre.

Algoritma 4

* Linear-time maximum contiguous subsequence sum algoritma.

```
int maxSubSum4(const vector<int>& a){
    int maxSum=0, thisSum=0;
    for(int j=0; j<a.size(); ++j){
        thisSum += a[j];
        if(thisSum > maxSum)
            maxSum = thisSum;
        else if(thisSum < 0)
            thisSum = 0;
    }
    return maxSum;
}
```

Bir for döngüsü olduğundan zaman karmaşıklığı O(n)'dir.

En iyi karmaşıklığa sahip olan algoritma

Algoritma 3'ün karmaşıklık Açıklaması:

* algoritmada recurrence yazılır:

sol yarıyı çözer → $T(n/2)$
sağ yarıyı çözer → $T(n/2)$
ortayı kontrol eder → n

Bu yüzden;

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

sadeleştir

* n/2 için ayrı formül yazılır:

$$T(n/2) = 2T(n/4) + n/2$$

* yerine koyulur → $T(n/2)$:

$$T(n) = 2(\underbrace{2T(n/4) + n/2}_{T(n/2)}) + n$$

* sadeleştirme:

$$T(n) = 4T(n/4) + 2n$$

* tekrar açınca pattern oluşur:

$$T(n) = 2^k T(n/2^k) + kn$$

başlangıç → dizi boyutu n

1. recursive = n/2
2. recursive = n/4
3. recursive = n/8

} k. recursive = $\underbrace{n/2^k}_{\text{alt problem boyutu}}$

• recursion bitişi:

her recursive çağrı dizinin bir parçası üzerinde çalışıyor. Bu parça artık tek eleman olunca:

alt problem boyutu ← $n/2^k = 1$

- $n = 2^k$
- $k = \log n$

$n \cdot n/2^k = 1 \cdot n \rightarrow 2^k = n$

$\hookrightarrow k = \log n$

* yerine koyma:

$$T(n) = \overset{2^k}{n}\,\underbrace{T(1)}_{T(n/2^k)} + n\,\underbrace{\log n}_{k}$$

$$T(n) = \underbrace{n + n\log n}_{\text{büyük olanı aldık yani } n\log n}$$

* big-O:

$$T(n) = O(n\log n)$$